

Progamação Concorrente

Teste de Avaliação

14 de maio de 2024

Duração: 2h00m

I

1 Explique porque a invocação concorrente de operações que envolvam vários objectos pode causar problemas, mesmo que cada objecto se proteja individualmente com código de controlo de concorrência interno. Ilustre o problema com um exemplo e descreva uma técnica de controlo de concorrência que o resolva.

2 Explique os motivos porque o `await()` numa `Condition` em Java deve ser efetuado com o `Lock` correspondente adquirido; nomeadamente porque o comportamento do `await()` foi pensado de modo a não ser o programador a ter que libertar o `Lock` antes do `await()`.

II

Pretende-se que escreva em Java, fazendo uso de primitivas baseadas em monitores, código que permita jogadores agruparem-se para participarem em partidas de futebol. Cada partida envolve 22 jogadores, cada um representado por uma thread, devendo 2 serem guarda-redes. Implemente a interface:

```
interface Matcher {  
    Match waitToPlay (int role) throws InterruptedException {  
    }  
}
```

A operação `waitToPlay()` é invocada quando um jogador pretende jogar, pela thread que o representa. Deverá bloquear até a partida poder começar (os 22 jogadores necessários, sendo 2 guarda-redes, querem jogar). O parâmetro `role`, indica qual o papel que o jogador vai desempenhar: 0 como guarda-redes e 1 como outro jogador. O método deverá devolver um objecto de uma classe `Match`, que se assume existir, que representa a partida envolvendo os 22 jogadores. Não há limite para o número de jogadores que pretendem jogar, devendo estes serem agrupados em sucessivas partidas, logo que possível, cada uma representada por um objeto `Match` diferente.

III

Apresente o código Erlang para implementar o mesmo serviço descrito no grupo II, através de um ou mais processos. Suponha que os clientes são processos Erlang que comunicam pelo mecanismo nativo de mensagens, e implemente também as funções de interface apropriadas para serem usadas por estes para interagirem com o(s) processos relevante(s).